

Joe's Blog

Random thoughts about programming and technology

Select the k-th smallest item from an unordered array in $O(n)$ in python

This blog contains two python implementations of the SELECT algorithm to select the k-th ordered item from an unsorted array of N items in $O(N)$ time. They derive from the discussion of RANDOM-SELECT and SELECT in the CLRS “Introduction to Algorithms, 3rd Edition” on pages 215-222 and from Professor David Eppstein’s [lecture notes](#) entitled “ICS 161: Design and Analysis of Algorithms” from January 30th, 1996.

Contents

- [Overview](#)
- [Implementation of select_random_pivot](#)
- [Implementation of select_median_of_medians_pivot](#)
- [Implementation of get_top_n_items](#)

Overview

Both implementations use a divide and conquer approach to recursively partition the input array into smaller and smaller chunks around a pivot point where each chunk contains data that honors an ordering relation to the pivot value until the k-th item matches the pivot. There are a lot of great discussions about how this works on the web and in text books (like CLRS). My goal here is not to try to improve on those discussions (I can’t). Instead, it is to simply show two working implementations.

The first algorithm ([select_random_pivot](#)), uses a random number to select the pivot point. The second algorithm ([select_median_of_medians_pivot](#)) uses a more deterministic approach to find a better approximation of the median.

The `select_random_pivot` algorithm has an average run time of $O(N)$ but a worst case run time of $O(N^2)$ in vanishingly rare cases. The `select_median_of_medians_pivot` algorithm has an average and worst case run time of $O(N)$.

These implementations were written to help me understand how the algorithms work. They were not designed for production use. For example, they do not do any bounds checking and they create sub arrays during recursion instead of operating on a single array in place which would probably be substantially faster but manipulating indices would, in my opinion, detract from the readability.

For a production environment you would want to create a several implementations, test them against characteristic datasets and choose the one that performs the best. When I tested these two implementations on a series of datasets, the `select_random_pivot` function won because it was at least 2X faster in every case.

To get a feel for how these algorithms might be used, I have included an example that shows how to get the top n ordered items from an unordered array in $O(N)$ time using the `get_top_n_items` function.

I hope that you find this helpful.

Implementation of `select_random_pivot`

The first implementation randomly chooses the pivot point. It is an implementation of the random select algorithm as described in CLRS “Introduction to Algorithms, 3rd Edition” on page 216 modified to partition the items in separate arrays.

The average run time is $O(N)$ and the worst case run time is $O(N^2)$ where N is the number of entries in the array but the probability of the worst case occurring in practice is vanishingly small.

select_random_pivot	Python
<pre>1 def select_random_pivot(array, k): 2 ''' 3 Implementation of the random select algorithm as described in CLRS 4 "Introduction to Algorithms, 3rd Edition" on page 216 modified to 5 partition the items in separate lists. 6</pre>	

```

7 This algorithm has a worst case run time of  $O(N^2)$  where N is the
8 number of entries in the array but the probability of that occurring
9 is vanishingly small. The average case is  $O(N)$ . It is normally much
10 faster than select_median_of_medians_pivot.
11
12 Here is how you might use it:
13
14 # Create a list of pseudo random numbers.
15 # Duplicates can occur.
16 num = 10000
17 array = [random.randint(1,1000) for i in range(num)]
18 random.shuffle(array)
19 random.shuffle(array)
20
21 # Get the value of the kth item.
22 k = 7
23 kval = select_random_pivot(array, k)
24
25 # Test it.
26 sorted_array = sorted(array)
27 assert sorted_array[k] == kval
28
29 @param array the list of values
30 @param k      k-th item to select.
31 @returns the k-th item
32 '''
33
34 # If the array is short, terminate the recursion and return the
35 # value without partitioning.
36 if len(array) <= 10:
37     array.sort()
38     return array[k]
39
40 # Randomly choose a pivot point.
41 # In vanishingly rare cases this can result in  $O(N^2)$ .
42 pivot_idx = random.randint(0, len(array) - 1)
43 pivot = array[pivot_idx] # pivot point value (not index)
44
45 # Now select recursively using the pivot.
46 # At this point we have the pivot. Use it to partition the input
47 # array into 3 categories: items that are less than the pivot
48 # value (array_lt), items that are greater than the pivot value
49 # (array_gt) and items that exactly equal to the pivot value
50 # (equals_array).
51 array_lt = []
52 array_gt = []
53 array_eq = []
54 for item in array:
55     if item < pivot:
56         array_lt.append(item)
57     elif item > pivot:
58         array_gt.append(item)
59     else:
60         array_eq.append(item)
61
62 # The array values have been partitioned according to their
63 # relation to the pivot value. The partitions look like this:
64 #
65 # +---+---+---+...+---+---+---+...+---+---+---+...
66 # | 0 | 1 | 2 |   | e | e+1 | e+2 |   | g | g+1 | g+2 |
67 # +---+---+---+...+---+---+---+...+---+---+---+...
68 #     array_lt      array_eq      array_gt
69 #
70 # If the value of k is in the range [0..e) then we know that
71 # the desired value is in array_lt so we need to recurse.

```

```

72     #
73     # If the value of k in the range [e..g) then we know that the
74     # desired value is in array_eq and we are done.
75     #
76     # If the value of k is >= g then we the desired value is in
77     # array_gt and we need to recurse but we also have to make sure
78     # that k is normalized with respect to array_gt so that it has the
79     # proper offset in the recursion. We normalize it by subtracting
80     # len(array_lt) and len(array_eq).
81     #
82     if k < len(array_lt):
83         return select_fct(array_lt, k)
84     elif k < len(array_lt) + len(array_eq):
85         return array_eq[0]
86     else:
87         normalized_k = k - (len(array_lt) + len(array_eq))
88         return select_fct(array_gt, normalized_k)

```

Implementation of select_median_of_medians_pivot

This implementation chooses a pivot point deterministically to reduce the worst case run time to $O(N)$. It is an implementation of the Blum, Floyd, Pratt, Rivest, Tarjan SELECT algorithm as described by David Eppstein. This algorithm is also discussed in CLRS “Introduction to Algorithms, 3rd Edition” on page 220.

This algorithm has worst case run time of $O(N)$ where N is the number of entries in the array.

Although this algorithm has better worst case performance than select_random_pivot, that algorithm is preferred because it is much faster in practice.

select_median_of_medians_pivot	Python
<pre> 1 def select_median_of_medians_pivot(array, k): 2 ''' 3 Implementation of the Blum, Floyd, Pratt, Rivest, Tarjan SELECT 4 algorithm as described by David Eppstein. 5 6 CITATION: http://www.ics.uci.edu/~eppstein/161/960130.html 7 8 This algorithm has worst case run time of $O(N)$ where N is the 9 number of entries in the array. 10 11 Although this algorithm has better worst case performance than 12 select_random_pivot(), that algorithm is preferred because it 13 is much faster in practice. 14 15 Here is how you might use it: 16 17 # Create a list of pseudo random numbers. </pre>	

```

18     # Duplicates can occur.
19     num = 10000
20     array = [random.randint(1,1000) for i in range(num)]
21     random.shuffle(array)
22     random.shuffle(array)
23
24     # Get the value of the kth item.
25     k = 7
26     kval = select_median_of_medians_pivot(array, k)
27
28     # Test it.
29     sorted_array = sorted(array)
30     assert sorted_array[k] == kval
31
32     @param array the list of values
33     @param k      k-th item to select.
34     '''
35
36     # If the array is short, terminate the recursion and return the
37     # value without partitioning.
38     if len(array) <= 10:
39         array.sort()
40         return array[k]
41
42     # Partition the array into subsets with a maximum of 5 elements
43     # each.
44     subset_size = 5 # max items in a subset
45     subsets = [] # list of subsets
46     num_medians = len(array) / subset_size
47     if (len(array) % subset_size) > 0:
48         num_medians += 1 # not divisible by 5
49     for i in range(num_medians):
50         beg = i * subset_size
51         end = min(len(array), beg + subset_size)
52         subset = array[beg:end]
53         subsets.append(subset)
54
55     # Find the medians in each subset.
56     # Note that it calls select_median_of_medians_pivot() recursively taking
57     # advantage of the fact that for len(array) <= 10, the select
58     # operation simply sorts the array and returns the k-th item. This
59     # could be done here but since the termination condition is
60     # required to get an infinite loop we may as well use it.
61     medians = [] # list of medians
62     for subset in subsets:
63         median = select_median_of_medians_pivot(subset, len(subset)/2)
64         medians.append(median)
65
66     # Now get the median of the medians recursively.
67     # Assign it to the local pivot variable because
68     # the pivot handling code is the same regardless
69     # of how it was generated. See select_random_pivot() for
70     # a different approach for generating the pivot.
71     median_of_medians = select_median_of_medians_pivot(medians, len(medians)/2)
72     pivot = median_of_medians # pivot point value (not index)
73
74     # Now select recursively using the pivot.
75     # At this point we have the pivot. Use it to partition the input
76     # array into 3 categories: items that are less than the pivot
77     # value (array_lt), items that are greater than the pivot value
78     # (array_gt) and items that exactly equal to the pivot value
79     # (equals_array).
80     array_lt = []
81     array_gt = []
82     array_eq = []

```

```

83     for item in array:
84         if item < pivot:
85             array_lt.append(item)
86         elif item > pivot:
87             array_gt.append(item)
88         else:
89             array_eq.append(item)
90
91     # The array values have been partitioned according to their
92     # relation to the pivot value. The partitions look like this:
93     #
94     #  +---+---+---+...+---+---+---+...+---+---+---+...
95     #  | 0 | 1 | 2 |   | e | e+1 | e+2 |   | g | g+1 | g+2 |
96     #  +---+---+---+...+---+---+---+...+---+---+---+...
97     #      array_lt      array_eq      array_gt
98     #
99     # If the value of k is in the range [0..e) then we know that
100    # the desired value is in array_lt so we need to recurse.
101    #
102    # If the value of k in the range [e..g) then we know that the
103    # desired value is in array_eq and we are done.
104    #
105    # If the value of k is >= g then we the desired value is in
106    # array_gt and we need to recurse but we also have to make sure
107    # that k is normalized with respect to array_gt so that it has the
108    # proper offset in the recursion. We normalize it by subtracting
109    # len(array_lt) and len(array_eq).
110    #
111    if k < len(array_lt):
112        return select_fct(array_lt, k)
113    elif k < len(array_lt) + len(array_eq):
114        return array_eq[0]
115    else:
116        normalized_k = k - (len(array_lt) + len(array_eq))
117        return select_fct(array_gt, normalized_k)

```

Implementation of get_top_n_items

This function shows how to get the top n ordered items from an unordered array using the select_random_pivot function. It demonstrates how to use the SELECT algorithms described here.

get_top_n_items	Python
<pre> 1 def get_top_n_items(array, n): 2 """ Get the top n items. """ 3 nth = select_random_pivot(array, n) # avg case: O(N), WC: O(N^2) 4 top_items = [] 5 6 # Start by getting all of the values less than the top value 7 # (O(N)). This guarantees that we don't miss some because there 8 # are duplicate nth values. 9 for value in array: 10 if value < nth: 11 top_items.append(value) 12 if len(top_items) == n: 13 return top_items # all done 14 15 # Now get the remaining values. </pre>	

```
16     # This is where duplicates are handled.
17     for value in array:
18         if value == nth:
19             top_items.append(value)
20             if len(top_items) == n:
21                 return top_items # all done
22
23     # We should never reach this point.
24     assert len(top_items) == n
```

📅 September 10, 2013 👤 jlinoff 📁 Algorithms, Programming, python

One thought on “Select the k-th smallest item from an unordered array in $O(n)$ in python”



jlinoff 👤

August 19, 2014 at 4:08 pm

Updated today to change “select_fct” to the correct function name based on a private comment that noted the use of the undefined “select_fct”. It was a typo.

You must be logged in to post a comment.

Proudly powered by WordPress